

```

static bool IsProperQueue(Queue<string> q1)
{
    Queue<string> q2 = new Queue<string>();
    char firstLett = ' ';
    while (!q1.IsEmpty())
    {
        firstLett = q1.Head()[0];
        while (!q1.IsEmpty() && q1.Head()[0] == firstLett)
            q1.Remove();
        while (!q1.IsEmpty())
        {
            if (q1.Head()[0] == firstLett) return false;
            q2.Insert(q1.Remove());
        }
        while (!q2.IsEmpty()) q1.Insert(q2.Remove());
    }
    return true;
}

```

```

static void insert(Queue<string> s, string str)
{
    char a = str[0];
    Queue<string> q = new Queue<string>();
    while (!s.IsEmpty() && a != s.Head()[0])
    {
        q.Insert(s.Remove());
    }
    q.Insert(str);
    //
    //
}

```

```

static void Fixit(Queue<string> s)
{
    Queue<string> q1 = new Queue<string>();
    Queue<string> q2 = new Queue<string>();
    while (!s.IsEmpty())
    {
        char c = s.Head()[0];
        while (!s.IsEmpty())
        {
            if (s.Head()[0] == c)
                q1.Insert(s.Remove());

            else
            {
                q2.Insert(s.Remove());
            }
        }
    }
}

```

```

    }
    while (!q2.IsEmpty()) s.Insert(q2.Remove());
}
while (!q1.IsEmpty())
    s.Insert(q1.Remove());
}
//static bool ismadrega(Node<int> n , int m) // לא גמור
//{
//    n=n.GetNext();
//    for (int i = 1; i <= m; i++)
//    {
//        for(int j = 0; j<i; j++)
//        {
//            if (n != null)
//            {
//                if (n.Getvalue() != i)
//                    return false;
//                n = n.GetNext();
//            }
//        }
//    }
//    return true;
//}

```

```

//public static bool IsSmaller(Queue<int> q1, Queue<int> q2)
//{
//    while(!q1.IsEmpty())
//    {
//        int max = q1.Remove();
//        while (!q1.IsEmpty())
//        {
//            if(max<q1.Head())
//                max = q1.Head();
//            q1.Remove();
//        }
//    }

//    while (!q2.IsEmpty())
//    {
//        int min = q2.Remove();
//        while (!q2.IsEmpty())
//        {
//            if (min > q2.Head())
//                min = q2.Head();
//            q2.Remove();
//        }
//    }

```

```
//}
```

```
static int maxdiff(Queue<int> s)
```

```
{
    int max = s.Head();
    int min = s.Head();
    s.Remove();
    while (!s.IsEmpty())
    {
        if (s.Head() < min)
            min = s.Head();
        if (s.Head() > max)
            max = s.Head();
        s.Remove();
    }
    return max - min;
}
```

```
static bool x(Queue<int> q, int num)
```

```
{
    Queue<int> ezer = new Queue<int>();
    while (!q.IsEmpty())
    {
        int val = q.Remove();
        Console.WriteLine(val);
        while (!q.IsEmpty())
        {
            int x = q.Remove();
            if (Math.Abs(val - x) == num)
                return true;
            ezer.Insert(x);
        }
        while (!ezer.IsEmpty()) q.Insert(ezer.Remove());
    }
    return false;
}
```

```
static void sortByAvg(Queue<int> q)
```

```
{
    int c = 0;
    int sum = 0;
    Queue<int> ezer = new Queue<int>();
    Queue<int> ezer2 = new Queue<int>();

    while (!q.IsEmpty())
    {
```

```

        c++;
        int val = q.Remove();
        sum += val;
        ezer.Insert(val);
    }
    double eveq = (double)sum / c;
    while (!ezer.IsEmpty())
    {
        int val = ezer.Remove();
        if (val < eveq)
            q.Insert(val);
        else
        {
            ezer2.Insert(val);
        }
    }
    while (!ezer2.IsEmpty()) q.Insert(ezer2.Remove());
}

```

```

static int LastLocisoin(Queue<int> q, int num)
{
    int c = 0;
    int courretLoc = 0;
    while (!q.IsEmpty())
    {
        c++;
        int val = q.Remove();
        if (val == num)
            courretLoc = c;
    }
    return courretLoc;
}

```

```

static void addOne(Queue<int> q)
{
    int c = 0;
    int courretLoc = 0;
    Queue<int> ezer = new Queue<int>();
    while (!q.IsEmpty())
    {
        c++;
        int val = q.Remove();
        if (q.IsEmpty() && val != 9)
            ezer.Insert(val + 1);
        else
            ezer.Insert(val);
        if (val == 9)
            courretLoc = c;
    }
}

```

```

        if (q.IsEmpty() && val == 9)
        {
            while (!ezer.IsEmpty())
            {
                int num = q.Remove();
                if (num != 9) q.Insert(num);
            }
        }
    }
    while (!ezer.IsEmpty()) q.Insert(ezer.Remove());
}

```

```

static Node<int> getToStuck(Node<int> n)
{
    Stack<int> stack = new Stack<int>();
    n = n.GetNext();
    Node<int> ezer = n;
    while (ezer != null)
    {
        stack.Push(ezer.Getvalue());
        ezer = ezer.GetNext();
    }
    Node<int> q = new Node<int>(-111);
    Node<int> ezer2 = q;
    int r = 1;
    while (!stack.IsEmpty())
    {
        int num = stack.Pop() + r;
        ezer2.SetNext(new Node<int>(num % 10, ezer2.GetNext()));
        r = num / 10;
    }
    if (r == 1)
    {
        ezer2.SetNext(new Node<int>(1, ezer2.GetNext()));
    }
    return ezer2;
} // 1 2 3 4 9 9 9

```

```

static Node<int> add2(Node<int> a, Node<int> b)
{
    a = a.GetNext();
    b = b.GetNext();
    Stack<int> a1 = new Stack<int>();
    Stack<int> b2 = new Stack<int>();
}

```

```

while (a != null)
{
    a1.Push(a.Getvalue());
    a = a.GetNext();
}
while (b != null)
{
    b2.Push(b.Getvalue());
    b = b.GetNext();
}
Node<int> result = new Node<int>(-111);
Node<int> ezer = result;
int r = 0;
while (!a1.IsEmpty())
{
    int num = a1.Pop() + b2.Pop() + r;
    ezer.SetNext(new Node<int>(num % 10, ezer.GetNext()));
    r = num / 10;
}
if (r == 1)
{
    ezer.SetNext(new Node<int>(r, ezer.GetNext()));
}
while (!b2.IsEmpty())
{
    ezer.SetNext(new Node<int>(b2.Pop(), ezer.GetNext()));
}
return result;
}

static bool ismadrega(Queue<int> q, int num)
{
    for (int i = 1; i <= num; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (q.IsEmpty()) { return false; }
            if (q.Remove() != i)
                return false;
        }
    }
    return q.IsEmpty();
}

static void delzogi(Node<int> n)
{

```

```

while (n.GetNext() != null)
{
    if (n.GetNext().Getvalue() % 2 == 0)
    {
        Node<int> ezer = n.GetNext();
        n.SetNext(n.GetNext().GetNext());
        ezer.SetNext(null);
    }
    else
    {
        n = n.GetNext();
    }
}
}
static void Show(Node<int> n)
{
    n = n.GetNext();
    while (n != null)
    {
        Console.Write(n.Getvalue() + " ");
        n = n.GetNext();
    }
    Console.WriteLine();
}
static Queue<int> madrega(int num)
{
    Queue<int> q = new Queue<int>();
    for (int i = 1; i <= num; i++)
    {
        for (int j = 0; j < i; j++)
        {
            q.Insert(i);
        }
    }
    return q;
}
static void delett(Node<int> node)
{
    Node<int> node1 = node.GetNext();
    while (node1 != null && node1.GetNext() != null && node1.GetNext().GetNext() != null)
    {
        Node<int> sec = node1.GetNext();
        Node<int> third = sec.GetNext();
        sec.SetNext(third.GetNext());
        third.SetNext(null);
        node1 = sec.GetNext();
    }
}

```

```

    }
}

public static Node<int> split(Node<int> chain)
{
    Node<int> newChain = new Node<int>(-111);
    Node<int> ezer = chain.GetNext();
    if (ezer.Getvalue() % 2 == 0)
    {
        ezer = ezer.GetNext();
        while (ezer != null)
        {
            if (ezer.Getvalue() % 2 == 0)
            {
                ezer = ezer.GetNext();
            }
            else
            {
                Node<int> current = ezer;
                newChain.SetNext(new Node<int>(current.Getvalue()));
                ezer.SetNext(null);
                ezer = current.GetNext();
            }
        }
    }
    else
    {
        ezer = ezer.GetNext();
        while (ezer != null)
        {
            if (ezer.Getvalue() % 2 == 1)
            {
                ezer = ezer.GetNext();
            }
            else
            {
                Node<int> current = ezer;
                newChain.SetNext(new Node<int>(current.Getvalue()));
                ezer.SetNext(null);
                ezer = current.GetNext();
            }
        }
    }
    Show(chain);
    Show(newChain);
    return newChain;
}

```



```

static int CountX(BinNode<int> b , int x)
{
    if(b == null) return 0;
    if(x == b.GetValue())
        return 1 + CountX(b.GetLeft() , x) + CountX(b.GetRight(),x);
    return CountX(b.GetLeft(), x) + CountX(b.GetRight(), x);
}

```

```

static bool isAllDiff(BinNode<int> b1 ,BinNode<int> b2)
{
    if(b1 == null) return true;
    if(CountX(b1,b2.GetValue()) > 1) return false;
    return isAllDiff(b1 ,b2.GetLeft()) && isAllDiff(b1,b2.GetRight());
}

```

```

static bool IsAllDiff(BinNode<int> b)
{
    return (isAllDiff(b, b));
}

```

```

static bool isGRt2(BinNode<int> b)
{
    return isGRt2(b, b);
}

```

```

static bool isGRt2(BinNode<int> b1 , BinNode<int> b2)
{
    if (b2 == null) return true;
    if(CountNum(b1,b2.GetValue()) < 2) return false;
    return isGRt2 (b1,b2.GetLeft()) && isGRt2(b1,b2.GetRight());
}

```

```

static int CountNum(BinNode<int> binNode , int x)
{
    if(binNode == null) return 0;
    if(binNode.GetValue() == x)
        return 1 + CountNum(binNode.GetLeft() , x) + CountNum(binNode.GetRight(),x);
    return CountNum(binNode.GetLeft(), x) + CountNum(binNode.GetRight(), x);
}

```

```

static void insertToBTS(BinNode<int> bts, int x)
{
    if (bts != null)
    {

```

```

        if (x < bts.GetValue())
            if (bts.GetLeft() == null)
                bts.SetLeft(new BinNode<int>(x));
            else
                insertToBTS(bts.GetLeft(), x);
        else
            if (bts.GetRight() == null)
                bts.SetRight(new BinNode<int>(x));
            else
                insertToBTS(bts.GetRight(), x);
    }
}

```

```

static BinNode<int> sortByfirst(int[] a)
{
    BinNode<int> b1 = new BinNode<int>(a[0]);
    for (int i = 1; i < a.Length; i++)
        insertToBTS(b1, a[i]);
    return b1;
}

```

```

static void swop(Node<int> b)
{
    Node<int> ezer2 = b;
    Queue<int> queue = new Queue<int>();
    while(b != null)
    {
        Node<int> ezer = b.GetNext();
        queue.Insert(b.GetValue());
        b.SetNext(null);
        b = ezer;
    }
    b = ezer2;
    b.SetNext(new Node<int>(queue.Remove(), b.GetNext()));
    while (!queue.IsEmpty())
    {
        b.SetNext(new Node<int>(queue.Remove(), b.GetNext()));
    }
    //Console.WriteLine(queue);
    Show(b);
}

```

```

static void swopNode(Node<int> b)
{
    Queue<int> queue = new Queue<int>();
    reversrec(b.GetNext(), queue);
}

```

```

static void reversrec(Node<int> b,Queue<int> q)
{
    if(b != null)
    {
        q.Insert(b.Getvalue());
        reversrec(b.GetNext(), q);
        b.Setvalue(q.Remove());
    }
}

```

```

static void swapQ(Queue<int> q)
{
    Stack<int> stack = new Stack<int>();
    while(q != null)
    {
        stack.Push(q.Remove() );
    }
    while(stack != null)
    {
        q.Insert(stack.Pop());
    }
}

```

```

static void xxx(Queue<int> q,Stack<int> s)
{
    if (!q.IsEmpty())
    {
        int val = q.Remove();
        s.Push(val);
        xxx(q, s);
        q.Insert(val);
    }
}

```

```

static void swaprec(Queue<int> q)
{
    Stack<int> s = new Stack<int>();
    xxx(q,s);
}

```

```

static string hatzpana(string s)
{
    string str = "";
    for(int i = 0; i < s.Length; i++)
    {
        if(s[i] != ' ')
            str += s[i];
    }
    char a = str[str.Length - 1];
}

```

```

string str2 = " ";
for(int i = 0; i < str.Length-1;i+=2)
{
    str2 += str[i+1];
    str2 += str[i];
}
str2 += (str2.Length % 2 == 1) ? a.ToString() : "";
string x = "";
for(int i = str2.Length - 1; i >= 0; i--)
{
    x += str2[i];
}
return x;
}
//static bool isG(int x, Queue<int> q)
//{
//    Queue<int> s = new Queue<int>();
//    int c = 0;
//    while (!q.IsEmpty())
//    {
//        int val = q.Remove()
//    }
//}

static Queue<int> aboveTwo(Queue<int> q)
{
    Queue<int> ezer = new Queue<int>();
    Queue<int> newQ = new Queue<int>();
    while (!q.IsEmpty())
    {
        int c = 1;
        int val = q.Remove();
        while (!q.IsEmpty())
        {
            int val2 = q.Remove();
            if (val == val2)
                c++;
            else
                ezer.Insert(val2);
        }
        if (c >= 2)
        {
            newQ.Insert(val);
        }
        while(!ezer.IsEmpty())
            q.Insert(ezer.Remove());
    }
    return newQ;
}

```

```

}
static Queue<int> madr(int x)
{
    Queue<int> queue = new Queue<int>();
    for(int i = 1; i <= x; i++)
    {
        for(int j = 0; j < i; j++)
        {
            queue.Insert(i);
        }
    }
    return queue;
}
static bool ismad(Queue<int> q , int x)
{
    for (int i = 1; i <= x; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (q.IsEmpty())
                return false;

            // מוציאים איבר ובודקים אם הוא שווה למספר הסדרה הנוכחי
            if (q.Remove() != i)
                return false;
        }
    }
    return q.IsEmpty();
}
static bool isSum(Node<int> node)
{
    node = node.GetNext();
    int sum = node.Getvalue();
    //Console.WriteLine(sum);
    node = node.GetNext();
    while (node != null)
    {
        if(sum != node.Getvalue())
            return false;
        sum += node.Getvalue();
        node = node.GetNext();
    }
    Console.WriteLine(sum);
    return true;
}
static bool isxxx(Node<int> node)
{

```

```

Node<int> x = node;
node = node.GetNext();
Stack<int> stack = new Stack<int>();
while (node != null)
{
    stack.Push(node.Getvalue());
    node = node.GetNext();
}
while (!stack.IsEmpty())
{
    x.Setvalue(stack.Pop());
    x=x.GetNext();
}
return isSum(x);
}
public static void First(Node<int> chain)
{
    chain = chain.GetNext();
    while(chain != null)
    {
        Node<int> node = new Node<int>(chain.Getvalue());
        node.SetNext(chain.GetNext());
        chain.SetNext(node);
        chain=chain.GetNext();
    }
}
public static void Second(Node<int> chain)
{
    Node<int> last = chain;
    while (last.GetNext() != null)
    {
        last = last.GetNext();
    }
    Node<int> first = chain.GetNext();
    Node<int> second = last;
    while (first != null)
    {
        Node<int> x = new Node<int>(first.Getvalue());
        last.SetNext(x);
        last= x;
        if (first == second)
            break;
        first=first.GetNext();
    }
}

//static bool equalSums(Stack<int> s)
//{

```

```

// int c = 0;
// Stack<int> ezer = new Stack<int>();
// Queue<int> queue = new Queue<int>();
// while (!s.IsEmpty())
// {
//     c++;
//     int val = s.Pop();
//     ezer.Push(val);
//     queue.Insert(val);
// }
// c /= 2;
// int copy = c + 1;
// while (c >= 0)
// {
//     int val = ezer.Pop();
//     s.Push(val);
//     c--;
// }
// int num = s.Top();      Console.WriteLine(num);

// while (!ezer.IsEmpty())
// {
//     s.Push(ezer.Pop());
// }
// Console.WriteLine(s);
// Console.WriteLine(queue);
// while (copy > 0)
// {
//     Console.WriteLine(s.Top()+ queue.Head());
//     if(s.Pop()+queue.Remove() != num) return false;
//     copy--;
// }
// return true;

```

//} // לא סופי

```

static int numNodesFollowing(Node<int> node)
{
    int c = 0;
    while (node.GetNext() != null)
    {
        c++;
        node = node.GetNext();
    }
    return c;
}
static bool isSection(Node<int> node)
{
    while(node != null)

```

```

    {
        if(numNodesFollowing(node) < node.Getvalue()) return false;
        node = node.GetNext();
    }
    return true;
}

```

```

//static void biluf(Node<int> node,int num)
//{
//    node = node.GetNext();
//    int copy = num;
//    while (node != null)
//    {
//        while (copy > 0)
//        {
//            int val = 0;
//            val += val *10 +node.Getvalue();
//            node = node.GetNext();
//            copy--;
//        }
//        int c = num -1;
//        while(c > 0)
//        {
//            Node<int> ezer = node;
//        }
//    }
// }
// }
//}

```

```

static void BuildNums(Node<int> n, int z)
{
    n = n.GetNext();
    while (n != null)
    {
        Node<int> e = n;
        int newNum = 0;
        int i = 0;
        while (i < z && e!=null)
        {
            newNum = newNum * 10 + e.Getvalue();
            i++;
            e = e.GetNext();
        }
        //Console.WriteLine(newNum);
        n.Setvalue(newNum);
        e = n.GetNext();
    }
}

```



```

Node<int> t = n;
i = 0;
while (i < z && t!= null)
{
    i++;
    t = t.GetNext();
}
n.SetNext(t);
n = t;
i = 1;
while (i < z && e!= null)
{
    i++;
    t = e.GetNext();
    e.SetNext(null);
    e = t;
}
}
}
static void insertToSortList(Node<int> n ,int x)
{
    while (n != null && n.Getvalue() < x )
    {
        n=n.GetNext();
    }
    Node<int> ezer = n.GetNext();
    n.SetNext(new Node<int>(x));
    n = n.GetNext();
    n.SetNext(ezer);
}
static bool IsN1InN2(Node<int> n1, Node<int> n2)
{
    int x1 = n1.Getvalue();
    while (n2 != null)
    {
        if (x1 == n2.Getvalue())
        {
            Node<int> p1 = n1.GetNext();
            Node<int> p2 = n2.GetNext();
            while (p1 != null && p2 != null && p1.Getvalue() == p2.Getvalue())
            {
                p1 = p1.GetNext();
                p2 = p2.GetNext();
            }
            if (p1 == null) return true;
        }
        n2 = n2.GetNext();
    }
}

```

```
}  
  return false;  
}
```